

Manual Técnico – TaskScript Analizador Léxico

Introducción

El presente documento constituye el Manual Técnico para el proyecto TaskScript Analyzer, un analizador léxico y sintáctico desarrollado en C++ utilizando el framework Qt para su interfaz gráfica. El propósito principal de este sistema es procesar un Lenguaje Específico de Dominio (DSL) diseñado para la definición y estructuración de tableros Kanban.

A través de la lectura de archivos con extensión .task, el sistema interpreta la configuración de columnas, tareas, prioridades, responsables y fechas límite. Este manual detalla la arquitectura subyacente, las decisiones de diseño, las estructuras de datos utilizadas, la gramática del lenguaje y los mecanismos de tolerancia a fallos. Está dirigido a desarrolladores, ingenieros de software y evaluadores académicos que deseen comprender el funcionamiento interno del compilador, mantener el código o escalar el proyecto en el futuro.

Arquitectura del Sistema

El sistema fue diseñado utilizando un patrón de Arquitectura en Capas (Pipeline o Tubería), típico en la construcción de compiladores modernos. Esta decisión arquitectónica permite una clara separación de responsabilidades (Separation of Concerns), donde la interfaz gráfica se comunica secuencialmente con las fases de análisis, y los datos fluyen en una sola dirección hasta culminar en la generación de reportes.

Componentes principales:

1. Frontend (GUI - Qt): Capa de presentación que maneja la interacción con el usuario. Provee el editor de texto integrado, las tablas de visualización en tiempo real y la invocación de los comandos del sistema.
2. Scanner (LexicalAnalyzer): Capa encargada de la tokenización. Convierte la cadena de caracteres de entrada en una secuencia finita de Tokens válidos, descartando espacios en blanco y saltos de línea innecesarios.
3. Parser (SyntaxAnalyzer): Capa central de validación estructural. Asegura que el orden de los tokens cumpla con la gramática definida y construye en memoria las estructuras de datos (DataRemember) y el Árbol de Análisis Sintáctico Abstracto (AST).
4. Módulo de Salida (ReportGenerator): Capa final que toma las estructuras en memoria y persiste la información generando archivos HTML renderizables y archivos .dot interpretables por Graphviz.

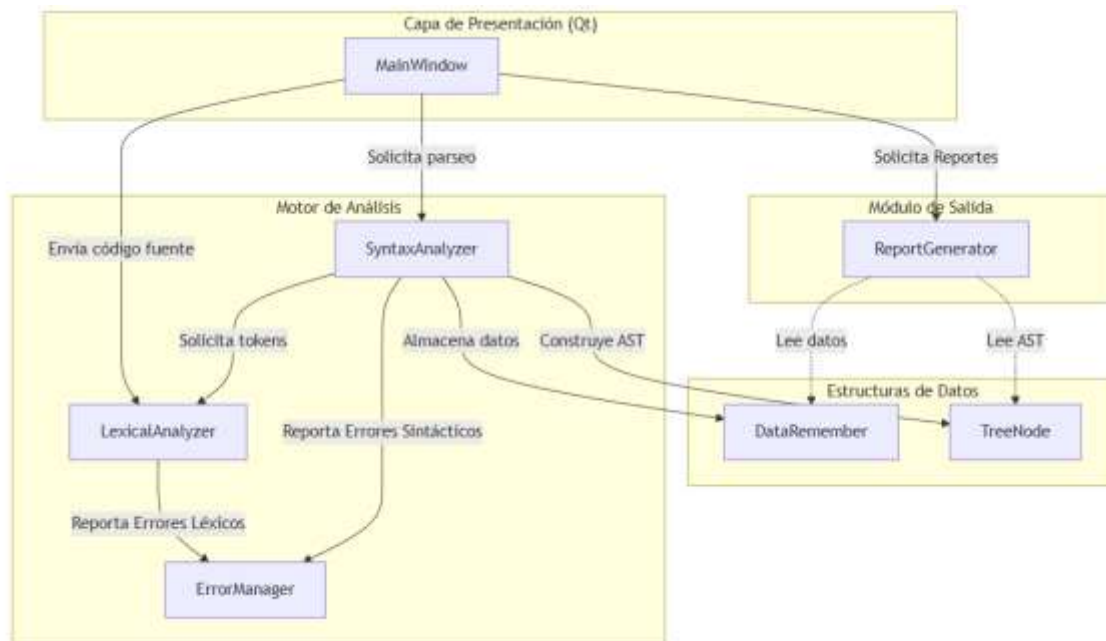
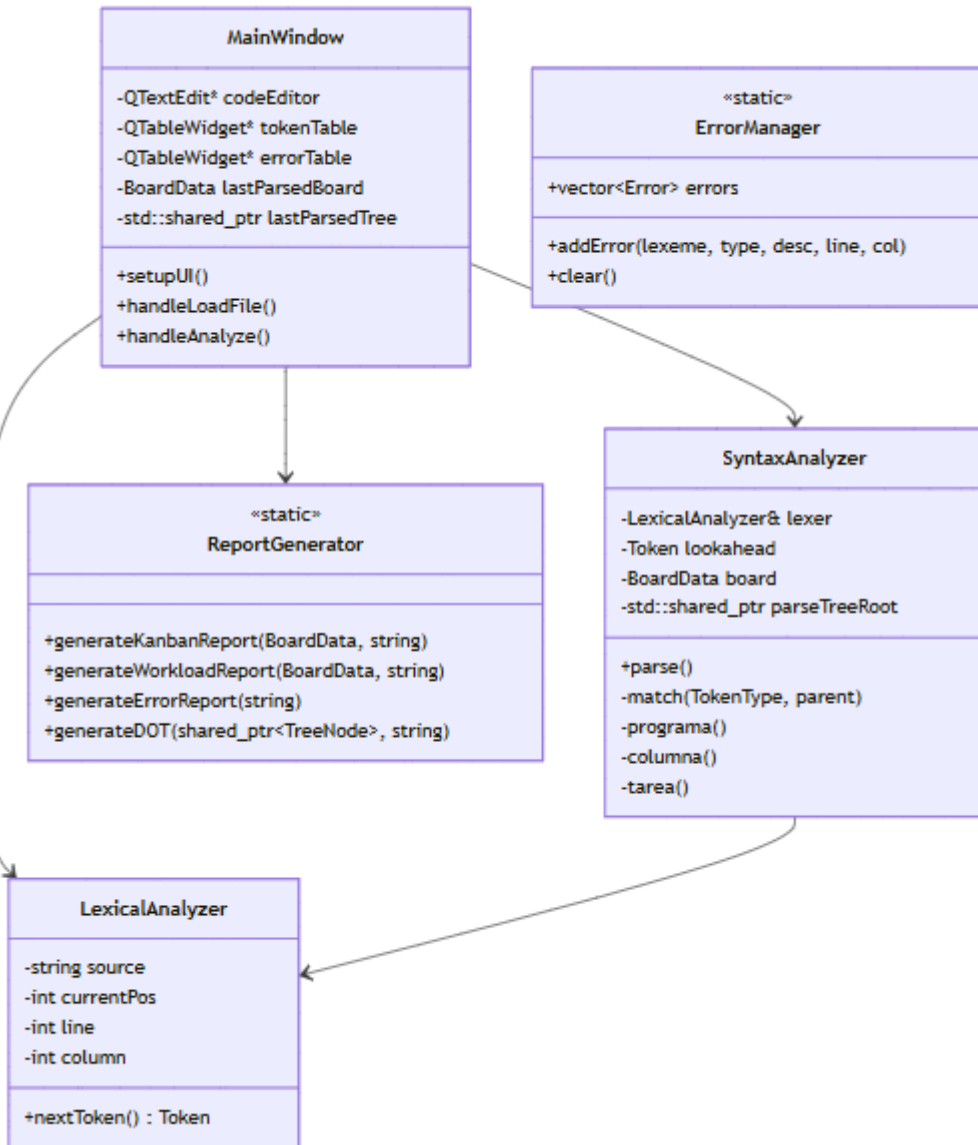


Diagrama de Clases Completo

El sistema utiliza Programación Orientada a Objetos (POO), priorizando la alta cohesión y el bajo acoplamiento para facilitar futuras expansiones (como agregar un analizador semántico puro o generación de código). La clase `MainWindow` actúa como orquestador, instanciando los analizadores solo cuando es necesario, mientras que clases como `ErrorManager` centralizan la recolección de datos anómalos.



Autómata Finito Determinista (AFD)

El analizador léxico (LexicalAnalyzer) no utiliza expresiones regulares externas, sino que implementa un autómata programado a medida. Este AFD procesa la entrada carácter por carácter, cambiando de estado según el símbolo leído.

Descripción de los Estados Principales:

- S0 (Estado Inicial): Punto de partida. Ignora espacios (), tabulaciones (\t) y saltos de línea (\n), ajustando los contadores de línea y columna. Lee el primer carácter válido para bifurcar al estado correspondiente.
- S1 (Identificadores y Reservadas): Inicia al detectar una letra. Itera leyendo caracteres alfanuméricos. Al encontrar un delimitador, compara el lexema acumulado contra un

diccionario interno para clasificarlo como palabra reservada (ej. tablero, tarea) o identificador.

- S2 (Cadenas de Texto): Se activa con el símbolo ". Captura cualquier carácter hasta encontrar la comilla de cierre. Incorpora validación de seguridad: si detecta un \n o el fin de archivo (EOF) antes de cerrar la cadena, transita a un estado de Error Léxico.
- S3 (Fechas): Valida la morfología numérica DDDD-DD-DD. Las transiciones requieren exactamente 4 dígitos, un guion, 2 dígitos, un guion, y 2 dígitos finales.
- S4-S10 (Símbolos Especiales): Estados de aceptación inmediata para terminales de un solo carácter ({, }, [,], :, ;, ,).

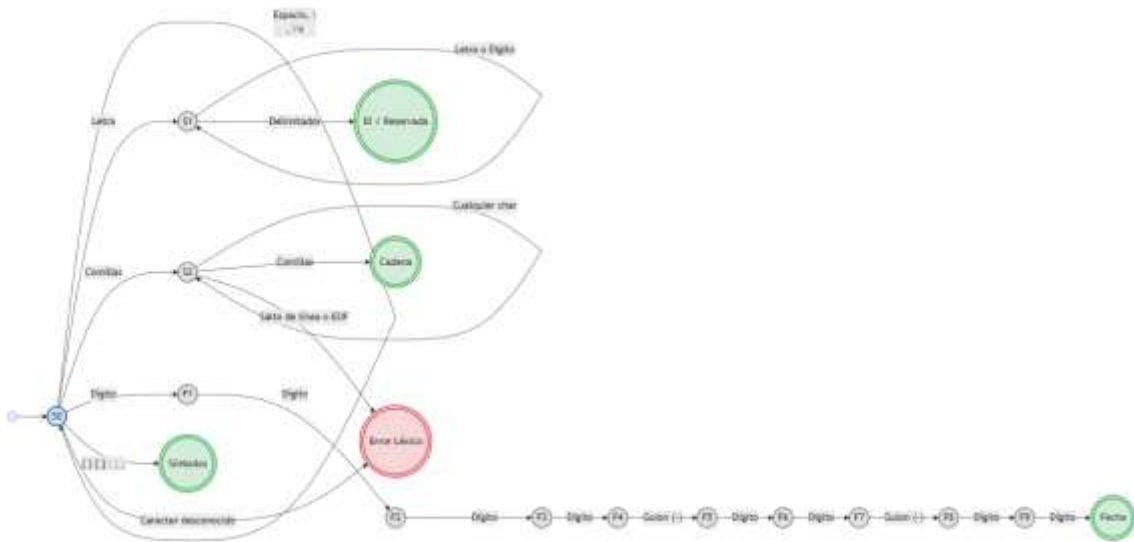


Tabla de transiciones del autómata

Estado Actual	Letra (a-zA-Z)	Dígito (0-9)	" (Comilla)	- (Guion)	Símbolos ({[]:;,)	Blancos (Espacio, \t)	\n (Salto) o EOF	Otro
S0 (Inicio)	S1	F1	S2	Err	AccSimbolos	S0	S0 / EOF	Err
S1 (ID)	S1	S1	AccID*	AccID*	AccID*	AccID*	AccID*	AccID*

Estado Actual	Letra (a-zA-Z)	Dígito (0-9)	" (Comilla)	' (Guion)	Símbolos ({>[]:;,)	Blancos (Espacio, \t)	\n (Salto) o EOF	Otro
S2 (Cadena)	S2	S2	AccCadena	S2	S2	S2	Err	S2
F1	Err	F2	Err	Err	Err	Err	Err	Err
F2	Err	F3	Err	Err	Err	Err	Err	Err
F3	Err	F4	Err	Err	Err	Err	Err	Err
F4	Err	Err	Err	F5	Err	Err	Err	Err
F5	Err	F6	Err	Err	Err	Err	Err	Err
F6	Err	F7	Err	Err	Err	Err	Err	Err
F7	Err	Err	Err	F8	Err	Err	Err	Err
F8	Err	F9	Err	Err	Err	Err	Err	Err
F9	Err	AccFecha	Err	Err	Err	Err	Err	Err

Gramática Libre de Contexto (GLC)

Para estructurar el lenguaje, se diseñó una gramática tipo LL(1) (Lectura de Izquierda a derecha, derivación por la Izquierda, con 1 token de anticipación). Esta gramática fue refactorizada expresamente para eliminar cualquier recursividad por la izquierda, garantizando que el árbol de derivación no entre en ciclos infinitos durante el análisis descendente.

Reglas de Producción (Notación BNF Modificada):

```
<programa> ::= TABLERO CADENA "{" <columnas> "}" [" ";""]  
  
<columnas> ::= <columna> <columnas_p>  
  
<columnas_p> ::= <columna> <columnas_p> | ε  
  
<columna> ::= COLUMN CADENA "{" <opt_tareas> "}" [" ";""]  
  
<opt_tareas> ::= <tareas> | ε  
  
<tareas> ::= <tarea> <tareas_p>  
  
<tareas_p> ::= "," <opt_tarea_siguiente> | ε  
  
<opt_tarea_siguiente> ::= <tareas> | ε  
  
<tarea> ::= TAREA ":" CADENA "[" <atributos> "]"  
  
<atributos> ::= <atributo> <atributos_p>  
  
<atributos_p> ::= "," <opt_atributo_siguiente> | ε  
  
<opt_atributo_siguiente> ::= <atributos> | ε  
  
<atributo> ::= PRIORIDAD ":" (ALTA | MEDIA | BAJA)  
                | RESPONSABLE ":" CADENA  
                | FECHA_LIMITE ":" FECHA
```

Descripción del Parser y Recuperación de Errores

Enfoque del Analizador Sintáctico

Se implementó un Analizador Sintáctico Descendente Predictivo Recursivo (Recursive Descent Parser). La fortaleza de este modelo radica en su correspondencia uno a uno con la gramática: cada producción no terminal se tradujo en un método de C++ (ej. programa(), columna()). El parser toma decisiones en tiempo real (predicciones) evaluando únicamente el token actual en espera (lookahead).

Estrategia de Recuperación de Errores (Panic Mode)

Para cumplir con los estándares de robustez, el compilador implementa una técnica de recuperación de errores tipo Panic Mode modificado a nivel de Token. El objetivo es nunca detener abruptamente la compilación. El flujo de recuperación funciona de la siguiente manera:

1. La función central de sincronización es `match(TokenType expected)`.
2. Si el lookahead actual no coincide con el token que exige la gramática, se detecta una asimetría.
3. El error se registra en el `ErrorManager` detallando la línea, columna, el lexema causante y una descripción de lo que se esperaba.
4. En lugar de lanzar una excepción que aborte el programa, el parser "consume" el token conflictivo llamando a `nextToken()` y retorna el control.
5. Esto fuerza al analizador a intentar sincronizarse con la entrada subsiguiente, permitiendo que el análisis continúe hasta el final del archivo en una sola pasada, mostrando al usuario un reporte exhaustivo de todos los fallos.

Justificación de Decisiones de Diseño

1. Uso de `std::shared_ptr` para el AST: En lenguajes como C++, la gestión manual de memoria en estructuras recursivas como árboles puede causar graves *memory leaks*. Al utilizar punteros inteligentes, se garantiza que los nodos (terminales y no terminales) se liberen automáticamente de la memoria una vez que el analizador termina su ciclo de vida o se carga un nuevo archivo.
2. Desacoplamiento Estructural (`DataRemember`): Para respetar el principio de Responsabilidad Única (SRP), el analizador sintáctico no genera los reportes de salida. Su único trabajo es validar y llenar las estructuras de datos ligeras (`BoardData`, `ColumnData`). Estas estructuras actúan como un puente (DTO) hacia el `ReportGenerator`.
3. Clases Estáticas para Gestión Global (`ErrorMessage`, `ReportGenerator`): Al tratarse de herramientas utilitarias y registros globales del sistema durante una pasada de compilación, se optó por métodos estáticos. Esto evitó la complejidad innecesaria de inyectar dependencias por todo el código o el uso del antipatrón Singleton.
4. HTML y CSS para Reportes: En lugar de generar reportes en texto plano, se optó por generar archivos `.html` con estilos embebidos. Esto mejora drásticamente la Experiencia de Usuario (UX) al permitir visualizar métricas con barras de progreso y tarjetas de colores (Kanban) sin requerir software adicional más allá de un navegador web estándar.

Conclusión

El desarrollo del TaskScript Analyzer demuestra la viabilidad de construir un intérprete funcional y robusto desde cero utilizando los fundamentos teóricos de los Lenguajes Formales. A través de la aplicación estricta de autómatas finitos y gramáticas libres de contexto predictivas, el sistema logra una validación de sintaxis precisa y una recuperación de errores resiliente.

El diseño modular y el desacoplamiento de las fases léxica, sintáctica y de generación gráfica permiten que el proyecto sea altamente escalable. En un futuro, la arquitectura actual facilitaría la incorporación de nuevas reglas, una fase de análisis semántico más profunda (por ejemplo, validar si un responsable tiene asignadas tareas en fechas contradictorias), o incluso la exportación directa de los datos a gestores de proyectos comerciales reales como Trello o Jira mediante APIs. La herramienta cumple eficazmente con su propósito, transformando un lenguaje de texto estructurado en información visual y métricas de alto valor agregado.